

Roteiro 2

Classificação supervisionada de intrusão

Dupla: Marinna Grigolli; Vitória Farias

Código da dupla: e4817f · random_state: 3833692159 · classe difícil: Infiltration + Heartbleed

1. Aquecimento: tabela-síntese (itens b, c, d)

Dataset aquecimento_autenticacao.csv (3.160 IPs, 8 sinais, 17,7% de ataque). intervalo_medio_seg fica em escala de milhares, bem acima das demais colunas (0–1 ou 1–3); essa diferença de escala é o motivo do experimento do item d).

Regra manual (item b): em vez de copiar o limiar de exemplo do enunciado, olhamos `aquecimento.groupby("label").describe()` para achar um corte que realmente separe as classes nesta base. Nenhum IP benigno tem `usuarios_distintos > 3` nem `usuarios_inexistentes > 1`; nos ataques a mediana é 26 e 10, respectivamente. Regra: `usuarios_distintos ≥ 4` OU `usuarios_inexistentes ≥ 2`.

Modelo	Acurácia	FN	FP	Recall ataque
Regra manual (item b)	0,9959	13	0	0,9768
Árvore d=3	0,9937	6	0	0,9643
Reg. logística (com scaler)	0,9831	16	0	0,9048
Reg. logística (sem scaler)	0,9947	3	2	0,9821

A regra manual foi calculada sobre o dataset inteiro (limiar fixo do item b), não só o treino; os outros três modelos respeitam o split 70/30 estratificado (`random_state=3833692159`). A regra recalibrada chega a 99,59%, na verdade melhor que a árvore e a regressão com scaler. Isso NÃO significa que a regra seja superior: ela só é tão boa porque foi calibrada olhando o `describe()` do dataset INTEIRO, já sabendo o rótulo de cada linha, uma forma de espiar o problema que não existe em produção. A árvore e a regressão aprendem a fronteira sozinhas a partir de só 70% dos dados (treino) e ainda generalizam bem para o teste: essa é a comparação que realmente importa.

Q1. Features da árvore nos 2 primeiros níveis: coincide com a regra do item b?

A árvore escolhe `usuarios_distintos` no 1º nível (limiar 3,5) e `usuarios_inexistentes` no 2º (limiar 1,5); falhas só aparece no 3º nível, como critério residual (limiar 15,5). Isso concorda muito bem com a regra recalibrada do item b) (limiares 4 e 2 para os mesmos dois sinais, achados de forma independente: a árvore via Gini, a regra via `groupby(label).describe()`). Bate também com a regressão logística: `usuarios_inexistentes` (coef. 6,48) e `usuarios_distintos` (coef. 6,10) são os dois coeficientes mais fortes, bem à frente de falhas (2,92), três abordagens independentes concordando nos mesmos dois sinais. A vantagem da árvore/regressão sobre a regra manual não está em achar sinais melhores (os três acharam os mesmos), e sim em aprender os limiares automaticamente a partir SÓ do treino, sem espiar o rótulo do dataset inteiro para calibrar os cortes à mão como fizemos na regra.

Q2. Regressão logística sem o StandardScaler: acurácia e `n_iter_`. Mecanismo?

A acurácia ficou parecida (até um pouco maior nesse teste específico: 99,47% sem scaler vs. 98,31% com scaler), mas o número de iterações do solver `lbfgs` saltou de 19 para 759 (quase 40x mais), embora ainda dentro do limite de `max_iter=1000` (sem disparar `ConvergenceWarning`). Mecanismo: sem padronização, `intervalo_medio_seg` (escala de milhares) domina numericamente o cálculo do gradiente em relação a features como `fracao_noturna` (escala 0–1), deixando a superfície de otimização mal-condicionada; o otimizador precisa de muitos passos pequenos para convergir, e os coeficientes finais deixam de ter escala comparável entre si (perdem interpretabilidade direta).

Q3. Por que `ip_origem` não pode ser feature?

Um endereço IP é só um identificador: não carrega, por si só, nenhum comportamento generalizável. Se entrasse no modelo, a árvore ou a regressão aprenderiam a decorar 'o IP X.X.X.X é malicioso' em vez do padrão de comportamento (muitas falhas, muitos usuários inexistentes etc.). No teste apareceriam IPs nunca vistos no treino, e o modelo não teria nada a dizer sobre eles: o comportamento aprendido seria memorização, não generalização, e o modelo ficaria inútil assim que o atacante trocasse de IP (trivial de fazer).

2. CICIDS2017: diagnóstico e armadilhas encontradas (item e)

Carregado com encoding='latin-1'. Shape bruto: 111.679 linhas × 83 colunas. Defeitos conhecidos confirmados: 65 colunas com espaço no nome, 102 linhas com ±Infinity ou NaN (coluna Flow Bytes/s), 0 duplicatas exatas (todas as colunas), 8 colunas constantes, 16 colunas idênticas a outra.

Além dos defeitos conhecidos, o enunciado avisava 3 problemas plantados. Os três foram encontrados seguindo as dicas do roteiro:

Armadilha	Deteção (código)	Impacto (dataset bruto)	Tratamento
1) alertas_ids	coluna fora do Anexo A + média por classe	1 coluna, todas as 111.679 linhas	coluna removida
2) vetores idênticos, rótulos diferentes	groupby(todas as features) .nunique(rótulo) > 1	145 grupos / 304 linhas	linhas removidas
3) quase-duplicatas mascaradas por contexto	duplicated(subset=features+ rótulo) vs. default	5.697 linhas em grupos; 4.530 'extras' a remover	drop_duplicates(subset= features+rótulo)

Armadilha 1: alertas_ids não pertence a nenhuma família do CICFlowMeter (Anexo A); sua média fica perto de 0 em BENIGN e salta para ~2,0–2,9 em qualquer classe de ataque, um sinal quase-binário só explicável como vazamento de rótulo (a coluna só poderia ter sido calculada depois de alguém já saber que o fluxo era malicioso). Um modelo treinado com ela chegaria perto de 100% de acurácia sem aprender nada sobre a rede, e desabaria em produção (onde esse 'alerta' não existiria antes da própria classificação).

Armadilha 2: o mesmo vetor de features aparece com rótulos diferentes (ex.: [BENIGN, DDoS]); nenhum classificador determinístico acerta os dois ao mesmo tempo; são exemplos com ruído irreduzível no rótulo (tipicamente fluxos muito curtos/degenerados, com assinatura estatística achatada).

Armadilha 3: drop_duplicates() padrão exige que TODAS as colunas sejam iguais, inclusive as de contexto (dia, turno, ordem_captura), que são metadados de quando a linha foi registrada, não do fluxo em si. O mesmo fluxo repetido com contexto diferente passa despercebido pelo duplicated() default; sem removê-lo, o split aleatório do item h) vazaria a 'mesma' observação entre treino e teste.

Nota de rigor: após as etapas anteriores da limpeza (remoção de Inf/NaN e da coluna alertas_ids) já terem alterado a população e as colunas de agrupamento, reaplicar as mesmas checagens DENTRO do pipeline sequencial remove 329 linhas (armadilha 2) e 6.030 linhas (armadilha 3), números diferentes de 304/4.530 porque medem o efeito da etapa em sequência, não o dataset bruto original. Ambos os números estão registrados no notebook; usamos os do dataset bruto (304/4.530) no parecer por serem a caracterização direta e reproduzível do problema na fonte.

Shape final após toda a limpeza (duplicatas exatas, Inf/NaN, 3 armadilhas, colunas constantes e colunas duplicadas): 105.218 linhas × 65 colunas.

3. Separabilidade (item f) e engenharia de features (item g)

Para cada classe de ataque, buscamos (só no treino) a feature e o limiar de uma árvore de profundidade 1 que atinge recall $\geq 0,9$ e precisão $\geq 0,5$ contra BENIGN:

Classe	Melhor feature	Recall	Precisão	Limiar	OK?
FTP-Patator	Destination Port	1,000	0,933	21,5	sim
SSH-Patator	Destination Port	1,000	0,806	23,0	sim
Heartbleed	Bwd Pkt Len Max	1,000	1,000	11584	sim
PortScan	Total Len Fwd Pkts	0,995	0,531	3,0	sim
DoS Hulk	Bwd Pkt Len Std	0,852	0,988	1545,3	não
DoS Slowhttptest	Flow IAT Mean	0,799	0,711	1,05e7	não
DoS GoldenEye	Bwd Pkt Len Std	0,664	0,984	1722,1	não
DDoS	Bwd Pkt Len Std	0,619	0,984	1489,9	não
DoS slowloris	Active Std	0,330	0,924	4,52e6	não
Infiltration	Total Len Fwd Pkts	0,231	0,857	282455	não
Web Attack-Brute Force	Destination Port	0,000	0,000	80,5	não
Bot	Destination Port	0,000	0,000	1840,5	não
Web Attack-Sql Inj.	Destination Port	0,000	0,000	80,5	não
Web Attack-XSS	Destination Port	0,000	0,000	80,5	não

Separáveis por 1 feature: FTP-Patator, SSH-Patator, Heartbleed, PortScan. Todos têm assinatura de rede extrema e fixa (porta de destino constante, resposta gigante no vazamento de memória, pacotes de sondagem quase vazios). As demais dependem de combinação de sinais (volume + tempo + variação de tamanho).

Classe difícil (Infiltration + Heartbleed): mesmo com a melhor feature combinada (Bwd Packet Length Max), recall de apenas 0,235; o histograma mostra a distribuição quase toda sobreposta a BENIGN. O motivo é estrutural: Heartbleed sozinho é quase perfeitamente separável, mas Infiltration não tem assinatura de flow-stats nenhuma (imita tráfego interno legítimo por design); juntar os dois num só rótulo faz qualquer feature que isole um atrapalhar o outro.

Engenharia de features (item g): 5 features derivadas, cada uma testada

fe_bytes_por_pacote, fe_razao_fwd_bwd_bytes, fe_header_payload_ratio, fe_idle_active_ratio, fe_flag_diversity: nenhuma isola sozinha a classe difícil (recall e precisão zerados no teste de limiar único), resultado esperado e documentado, já que a classe difícil mistura dois mecanismos opostos de ataque. Ainda assim, 2 delas (fe_header_payload_ratio e fe_bytes_por_pacote) aparecem no top-15 de importância do Random Forest (item i), ajudando em combinação com outras features, mesmo não resolvendo isoladamente.

Q4. Armadilhas encontradas: qual a mais perigosa e por que a acurácia seria mentira?

As 3: alertas_ids (vazamento de rótulo), vetores idênticos com rótulos diferentes (304 linhas), quase-duplicatas mascaradas por contexto (4.530 linhas). A mais perigosa é alertas_ids: por ser derivada do próprio rótulo, um modelo treinado com ela chegaria perto de 100% de acurácia. Esse número seria mentira porque, em produção, esse 'alerta' não existe antes da própria classificação: o modelo estaria lendo a resposta da prova, não aprendendo padrões de tráfego generalizáveis.

4. Random Forest: split aleatório × temporal (itens h, i, j)

Split	Acurácia	Macro-F1	Recall Infiltr.	Recall Heartbl.	Recall BENIGN
Aleatório 70/30	0,9891	0,9228	0,833	1,000	0,9985
Temporal (campanha)	0,9388	0,7973	0,833	1,000	0,9730

Recall por classe, aleatório × temporal (ordenado pela maior queda):

Classe	Recall aleat.	Recall temp.	Queda
Bot	0,978	0,155	0,823
Web Attack - Sql Injection	0,833	0,143	0,690
DoS slowloris	0,995	0,832	0,163
DoS Hulk	0,996	0,871	0,125
DoS Slowhttptest	0,993	0,868	0,125
Web Attack - XSS	0,359	0,313	0,045
Web Attack - Brute Force	0,790	0,754	0,036
DoS GoldenEye	0,992	0,960	0,033
BENIGN	0,999	0,973	0,026
SSH-Patator	0,995	0,987	0,008
FTP-Patator	0,998	0,993	0,005
DDoS	0,992	0,988	0,004
PortScan	0,989	0,988	0,002
Heartbleed / Infiltration	1,000 / 0,833	1,000 / 0,833	0,000

feature_importances_ (item i): nenhuma feature domina isoladamente (maior importância ~0,05, sem nada acima de 0,3), o que confirma que a limpeza do item e) funcionou. Se alertas_ids ainda estivesse lá, dominaria essa lista sozinha. fe_header_payload_ratio e fe_bytes_por_pacote (item g) aparecem no top-15.

Q5. Classes separáveis por 1 feature × classe difícil: diferença em termos de rede

Separáveis: FTP-Patator, SSH-Patator, Heartbleed, PortScan. Não separáveis: DoS (Hulk/Slowhttptest/GoldenEye/slowloris), DDoS, Infiltration, Web Attack (Brute Force/Sql Injection/XSS), Bot. As separáveis têm uma assinatura de protocolo fixa e extrema que nenhum tráfego normal replica (porta constante, pacote de resposta gigante, sondagem quase vazia). A classe difícil mistura um ataque com assinatura fortíssima (Heartbleed) com outro sem assinatura de flow-stats nenhuma (Infiltration imita tráfego interno legítimo por design); nenhum corte único resolve o par junto.

Q6. Classe que mais caiu no temporal: evidência e mudança no comportamento do atacante

Bot: de recall 0,978 (aleatório) para 0,155 (temporal), queda de 0,823, a maior e mais robusta (quase 2.000 linhas, concentradas na sexta-feira). Medianas início × fim da campanha: Flow Duration 58.821 → 1.002.738 (17x maior); Fwd Packet Length Mean 42,4 → 0,0 bytes; Flow Bytes/s 131.868 → 17,9; Flow Packets/s 1.046 → 6,0. O atacante muda de um 'check-in' ruidoso e ativo (curto, com dados de verdade) para 'beaconing' silencioso (conexões longas, quase sem payload), padrão clássico de C2 que o modelo, treinado só no início, nunca aprendeu a reconhecer.

5. Questões finais e parecer contra a produção (item k)

Q7. O split temporal é injusto com o modelo, ou é a única avaliação honesta?

É a única honesta. Em produção o modelo é treinado com o histórico disponível e testado contra tráfego futuro nunca visto; o split aleatório embaralha início e fim da mesma campanha entre treino e teste, deixando o modelo 'espiar' um pedaço do padrão final durante o treino, o que não existe na vida real, onde o futuro é por definição desconhecido no momento do treino. A queda de 98,9% para 93,9% de acurácia (e do recall do Bot, de 0,98 para 0,16) mostra que boa parte do desempenho 'aleatório' era memorização de um recorte específico da campanha, não capacidade real de generalizar.

Q8. A feature do item j (`fe_iat_por_byte`) ajudou?

Não ajudou: o recall de Infiltration no split temporal ficou idêntico (0,833, os mesmos 2 de 12 fluxos errados) antes e depois de acrescentar a feature, que ficou entre a posição 26 e 27 de 67 em importância (varia ± 1 posição entre execuções do Random Forest). Isso revela um limite estrutural: com apenas 38 exemplos de Infiltration em todo o dataset (12 no teste temporal), engenharia de features não cria informação que não está nos dados. Os 2 fluxos que erram caem numa região tão rara e extrema do espaço de features (`fe_iat_por_byte` e Flow IAT Mean muito acima até da mediana dos próprios acertos de Infiltration) que nenhuma razão calculada a partir das mesmas colunas resolve; falta exemplos de treino cobrindo esse extremo, não falta uma feature melhor.

Parecer técnico: sobre colocar o modelo em produção na segunda-feira

Recomendo NÃO aprovar o deploy do modelo atual no IDS com base apenas na acurácia de split aleatório. Cinco pontos, cada um ancorado num número deste notebook:

- A acurácia de 98,9% do split aleatório (item h) não sobrevive ao teste honesto. No split temporal, que simula produção (treinar com o passado, testar no futuro), a acurácia cai para 93,9% e o macro-F1 despenca de 0,923 para 0,797. Parte do desempenho 'aleatório' é memorização, não generalização.
- O recall de Bot desaba de $\sim 0,98$ para $\sim 0,16$ no split temporal (item h). Em produção, um C2 de botnet que muda de comportamento ao longo da campanha passaria despercebido em $\sim 84\%$ das vezes depois da fase inicial: o IDS ficaria cego justamente na fase mais longa e discreta do ataque.
- O experimento do item l) (treinar seg-qui, testar sex) mostra o pior cenário possível: ataque nunca visto antes. Bot foi classificado como BENIGN em 100% dos casos; DDoS foi confundido majoritariamente com DoS Hulk. Um ataque novo, ou vira tráfego normal (o pior erro para um IDS), ou vira um ataque diferente do que é (atrapalha a resposta a incidente).
- O dataset de origem tinha 3 armadilhas que só apareceram investigando, não olhando `describe()` (item e). Uma delas (`alertas_ids`) era vazamento de rótulo puro: se não removida, os 98,9% citados pelo gestor poderiam estar inflados por uma coluna que não existiria em produção.
- 304 linhas do dataset bruto tinham o mesmo vetor de features com rótulos diferentes, e outras 4.530 eram cópias quase-idênticas do mesmo fluxo mascaradas pelas colunas de contexto (item e). Mesmo o rótulo 'verdade' do CICIDS2017 tem ruído: toda métrica de acurácia carrega uma margem de erro que a média de 98% nunca comunica ao gestor.

Conclusão: o modelo pode ser um bom ponto de partida, mas precisa ser reavaliado com split temporal, monitorado por classe (não só por acurácia agregada) e testado especificamente contra padrões de ataque que mudam de comportamento ao longo do tempo antes de entrar em produção.

6. Ataques nunca vistos (item I) e Anexo B

Random Forest treinado apenas seg-qui (66.687 fluxos), testado na sexta (38.531 fluxos, dia='sex'). Bot nunca aparece no treino; DDoS (42), Infiltration (38), PortScan (26) têm menos de 50 exemplos.

Bot (n=1.971): 100% classificado como BENIGN. PortScan (n=8.154): 8.142 viram BENIGN. DDoS (n=8.583): 3.241 viram BENIGN e 5.342 viram DoS Hulk. Um classificador supervisionado só reconhece o que já viu rotulado no treino: contra um ataque genuinamente novo, ele só absorve na classe majoritária (BENIGN, o silêncio mais perigoso) ou empresta o rótulo do ataque conhecido mais parecido (DDoS → DoS Hulk, que ao menos dispara um alerta, só que errado). Não é defeito de implementação: é limitação estrutural de classificação supervisionada fechada. Um IDS de produção precisa de detecção de anomalia complementar e retreino frequente.

Anexo B: Ficha de resultados

Dupla: Marinna Grigolli; Vitória Farias Data: 13/09/2026

Aquecimento

Modelo	Acurácia	FN	FP	Recall ataque
Regra manual (item b)	0,9959	13	0	0,9768
Árvore d=3	0,9937	6	0	0,9643
Reg. logística (com scaler)	0,9831	16	0	0,9048
Reg. logística (sem scaler)	0,9947	3	2	0,9821

Features do 1º e 2º nível da árvore: usuarios_distintos (1º, limiar 3,5); usuarios_inexistentes (2º, limiar 1,5). Regra manual: usuarios_distintos ≥ 4 OU usuarios_inexistentes ≥ 2 (limiares achados via groupby(label).describe(), não copiados do enunciado).

CICIDS2017

Código da dupla: e4817f random_state: 3833692159 classe difícil: Infiltration + Heartbleed

Armadilha / passo de limpeza	Linhas (ou colunas) afetadas
Inf / NaN	102 linhas
Duplicatas exatas	0
Armadilha 1: alertas_ids	1 coluna (111.679 linhas)
Armadilha 2: vetores contraditórios	304 linhas / 145 grupos
Armadilha 3: quase-duplicatas mascaradas	4.530 linhas
Total final (linhas × colunas)	105.218 × 65

	Acurácia	Macro-F1	Recall classe difícil	Recall BENIGN
Split aleatório	0,9891	0,9228	0,833 (Inf.) / 1,000 (Heart.)	0,9985
Split temporal	0,9388	0,7973	0,833 (Inf.) / 1,000 (Heart.)	0,9730

Classes separáveis por uma feature: FTP-Patator, SSH-Patator, Heartbleed, PortScan.

Classe que mais caiu no temporal: Bot, de 0,978 para 0,155.

Feature do item j: fe_iat_por_byte. Efeito no recall temporal (Infiltration): 0,833 → 0,833 (sem efeito).